

# Logic in AI : Sat Solvers: WalkSAT

## Algorithm - Part - 8

### Introduction

In the realm of Artificial Intelligence, solving Boolean Satisfiability (SAT) problems is a foundational challenge with widespread applications, from hardware verification to automated planning. While systematic search algorithms like DPLL offer completeness, their computational demands can quickly become prohibitive for large-scale instances. This is where local search algorithms emerge as powerful, often indispensable, tools. Unlike their systematic counterparts, local search methods do not guarantee finding a solution or proving unsatisfiability, but they excel at finding solutions to very large, satisfiable problems with remarkable efficiency.

This lecture delves into the mechanics of local search for SAT, specifically focusing on the evolution from basic concepts to the highly effective WalkSAT algorithm. We'll explore how these algorithms operate on complete truth assignments, iteratively refining them to minimize the number of unsatisfied clauses. The practical significance of these techniques is profound: while a state-of-the-art DPLL algorithm might tackle N-Queens problems with up to 700 variables, local search algorithms like WalkSAT can routinely solve problems involving over 100,000 variables. This dramatic difference in scale underscores why understanding and applying local search is critical for tackling real-world AI challenges.

### Core Concepts

| Term                      | Definition                                                                                                                                                                     | Significance                                                                                                                                                                |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local Search              | An optimization technique that iteratively improves a candidate solution by making small, local changes until a satisfactory solution is found or a stopping criterion is met. | Essential for solving large-scale SAT problems where systematic search is computationally infeasible, offering practical solutions despite lacking completeness guarantees. |
| Complete Truth Assignment | A state where every propositional variable in a SAT problem is assigned a                                                                                                      | The fundamental unit of exploration for local search SAT solvers, as they operate                                                                                           |

| Term                         | Definition                                                                                                                                                                      | Significance                                                                                                                                    |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
|                              | definite truth value (true or false).                                                                                                                                           | by modifying these assignments.                                                                                                                 |
| <b>Neighborhood Function</b> | A rule that defines the set of possible next states (assignments) reachable from the current state by making a single, small change, such as flipping a variable's truth value. | Determines the search space connectivity and the types of moves an algorithm can make to explore new assignments.                               |
| <b>Objective Function</b>    | A quantitative measure used to evaluate the quality of a given truth assignment, typically by counting the number of satisfied versus unsatisfied clauses.                      | Guides the local search process, indicating whether a move improves the current assignment towards a solution (minimizing unsatisfied clauses). |
| <b>Random Walk</b>           | A search strategy where, with a certain probability, a variable is chosen randomly from an unsatisfied clause and its truth value is flipped.                                   | Helps escape local optima by introducing non-greedy exploration, preventing the search from getting stuck in suboptimal regions.                |
| <b>Greedy Move</b>           | A search strategy where the variable flip that yields the greatest improvement in the objective function (e.g., minimizes unsatisfied clauses the most) is chosen.              | Drives the search towards better solutions efficiently, exploiting local gradients in the objective function.                                   |
| <b>GSAT</b>                  | A foundational local search SAT solver that combines probabilistic random walks with greedy choices to find satisfying assignments.                                             | A precursor to more advanced local search algorithms like WalkSAT, demonstrating the power of combining random and greedy strategies.           |
| <b>WalkSAT</b>               | An advanced local search SAT solver that refines GSAT by focusing on efficiency per                                                                                             | Became a de facto standard for local search SAT solving due to its superior practical                                                           |

| Term                  | Definition                                                                                                                 | Significance                                                                                                                         |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
|                       | move, particularly by evaluating variables only within a chosen unsatisfied clause.                                        | performance and ability to handle extremely large problems.                                                                          |
| <b>Make Function</b>  | Quantifies the number of currently false clauses that would become true if a specific variable's truth value were flipped. | Helps evaluate the positive impact of a potential variable flip on the overall satisfiability of the formula.                        |
| <b>Break Function</b> | Quantifies the number of currently true clauses that would become false if a specific variable's truth value were flipped. | Helps evaluate the negative impact of a potential variable flip, indicating how many previously satisfied clauses would be "broken." |

## Detailed Analysis

### 1. Local Search Fundamentals for SAT Solving

Local search algorithms for SAT problems operate on a fundamentally different principle than systematic methods. Instead of building a partial assignment, they always maintain a **complete truth assignment** for all variables. The core idea is to start with an arbitrary assignment and then iteratively modify it to reduce the number of unsatisfied clauses until a satisfying assignment (where all clauses are true) is found.

The process typically involves:

- **Initialization:** Begin with a randomly generated complete truth assignment.
- **Neighborhood Exploration:** Define a **neighborhood function** that specifies how to move from the current assignment to a neighboring one. For SAT, this usually means flipping the truth value of a single variable.
- **Objective Evaluation:** Use an **objective function** to assess the quality of the current assignment. This function typically counts the number of satisfied clauses or, conversely, the number of unsatisfied clauses. The goal is to maximize satisfied clauses or minimize unsatisfied ones.
- **Move Selection:** Choose the next assignment based on a strategy that balances exploration and exploitation. Common strategies include:
  - **Hill Climbing:** Always move to a neighboring state that improves the objective

function.

- **Random Walk:** Randomly select a variable to flip, often to escape local optima.
- **Random Restart:** Periodically abandon the current search path and restart from a new random assignment to avoid getting stuck in deep local optima.

A common hybrid strategy, exemplified by algorithms like GSAT, combines greedy moves with random walks. For instance, with a certain probability  $p$ , the algorithm might flip any random variable within an unsatisfied clause. With probability  $1-p$ , it might choose the "best" variable to flip, where "best" is defined as the one that minimizes the number of unsatisfied clauses after the flip. This blend allows for both directed improvement and stochastic exploration. The practical power of these methods is evident in their ability to solve problems with hundreds of thousands of variables, far exceeding the capacity of systematic solvers for many real-world instances.

## 2. Quantifying Flip Impact: Make and Break Functions

To make informed decisions about which variable to flip, local search algorithms need a way to evaluate the potential impact of each flip. This evaluation is captured by two key concepts: the `make function` and the `break function`.

- **Make Function** : When a variable  $x$  is flipped, the `make(x)` value represents the number of clauses that were previously false (unsatisfied) but become true (satisfied) as a direct result of flipping  $x$ . This signifies a positive contribution to the objective function.
- **Break Function** : Conversely, the `break(x)` value represents the number of clauses that were previously true (satisfied) but become false (unsatisfied) if  $x$  is flipped. This signifies a negative consequence of the flip.

The overall change in the number of unsatisfied clauses when flipping variable  $x$  from assignment  $s_1$  to  $s_2$  can be understood as:

$$\text{Unsatisfied}(s_2) = \text{Unsatisfied}(s_1) - \text{make}(x) + \text{break}(x)$$

The goal of any flip is to maximize `make(x)` and minimize `break(x)`. Ideally, we want to find a flip where `make(x)` is high and `break(x)` is low, or even zero.

## 3. The WalkSAT Algorithm: Efficiency Through Focused Evaluation

WalkSAT refines the ideas introduced by GSAT by incorporating two crucial insights that significantly improve its practical performance:

1. **Prioritizing "Free" Moves:** If a variable flip `breaks nothing` (i.e., `break(x) = 0`), it is an unequivocally good move. Such a flip will either

maintain the current number of satisfied clauses or increase it, without any negative side effects. WalkSAT prioritizes finding and executing such moves.

2. **The "Near Optimum" Observation:** When a local search algorithm is close to a local optimum (meaning the number of unsatisfied clauses is small and the problem is tightly constrained), most variable flips tend to `make` only one clause true. If a flip could make many clauses true, it would likely have been performed much earlier in the search. Therefore, near optima, the `make` count is often 1, and the critical factor for choosing a flip becomes minimizing the `break` count.

Based on these insights, the WalkSAT algorithm proceeds as follows:

1. **Select an Unsatisfied Clause:** Randomly pick one clause `C` that is currently false (unsatisfied) in the current truth assignment. This is a crucial difference from GSAT, which might consider all unsatisfied clauses.
2. **Evaluate Variables within the Clause:** For each variable `x` that appears in the chosen clause `C`, calculate its `break(x)` value. This means determining how many currently satisfied clauses would become unsatisfied if `x` were flipped.
3. **Prioritize Zero-Break Moves:** Check if there exists any variable `x` within clause `C` for which `break(x) = 0`.
4. If such a variable `x` exists, immediately flip `x`. This is a "free" move that guarantees no regression in the number of satisfied clauses.
5. **Probabilistic Choice (if no zero-break move):** If all variables in clause `C` would break at least one satisfied clause if flipped, then WalkSAT makes a probabilistic decision:
6. With probability `p` (a parameter, e.g., 0.5), choose a random variable `x` from clause `C` and flip it (a random walk step).
7. With probability `1-p`, choose the variable `x` from clause `C` that has the minimum `break(x)` value (a greedy step) and flip it.
8. **Random Restarts:** Like many local search algorithms, WalkSAT is typically embedded within a random restart loop. After a certain number of flips or if the search appears stuck, the algorithm restarts from a new random assignment to explore different parts of the search space. Parameters like `p`, maximum flips before restart, and the total number of restarts are critical for tuning performance.

## 4. WalkSAT's Efficiency Advantage Over GSAT

The primary advantage of WalkSAT over GSAT lies in its computational efficiency per move.

- **GSAT's Evaluation Cost:** GSAT, especially in its greedy step, often needs to iterate through all unsatisfied clauses and all variables within them to find the single "best" variable to flip (the one that minimizes the total number of unsatisfied clauses). This global evaluation can be computationally expensive, especially for problems with many clauses and variables.
- **WalkSAT's Focused Evaluation:** WalkSAT, by contrast, first picks a single random unsatisfied clause. It then restricts its evaluation of `break` values to only the variables within that specific clause. This significantly reduces the number of evaluations required for each move. While this might mean that WalkSAT doesn't always pick the globally "best" variable, its ability to perform many more moves per unit of time often leads to finding solutions much faster in practice.
- **Guaranteed Progress:** By focusing on an unsatisfied clause, WalkSAT ensures that the chosen clause becomes true after the flip, even if other clauses might become false. This guarantees that the algorithm is always making progress towards satisfying at least one currently unsatisfied clause.

This efficiency makes WalkSAT a much faster algorithm in practice compared to GSAT. Its ability to perform a large number of search steps quickly, even if each step is not optimally greedy, has made it the de facto standard for solving large SAT formulas using local search techniques.

### Key Takeaways

- [ ] Local search algorithms are essential for solving large-scale SAT problems, often outperforming systematic methods like DPLL for satisfiable instances.
- [ ] These algorithms operate on complete truth assignments, iteratively modifying them to minimize unsatisfied clauses.
- [ ] GSAT combines random walks and greedy moves, flipping variables to improve the assignment or escape local optima.
- [ ] The 'make' and 'break' functions quantify the positive and negative impacts of a variable flip on clause satisfaction.
- [ ] WalkSAT refines GSAT by prioritizing moves that do not 'break' any satisfied clauses and by focusing variable evaluations within a single randomly chosen unsatisfied clause.

- [ ] WalkSAT's primary advantage is its significantly reduced computational cost per move, allowing it to perform many more search steps quickly.
- [ ] The algorithm uses a probabilistic choice between a random flip and a greedy flip (minimizing breaks) for variables within the selected unsatisfied clause.
- [ ] Random restarts are a critical component of WalkSAT, enabling it to escape local optima and explore diverse regions of the search space.
- [ ] WalkSAT's practical speed and effectiveness on large problem instances have established it as a de facto standard for local search SAT solving.

## **Conclusion**

The journey from systematic SAT solvers to the sophisticated local search algorithms like WalkSAT highlights a fundamental trade-off in AI problem-solving: completeness versus efficiency. While systematic methods offer guarantees, their exponential complexity often renders them impractical for the massive problem instances encountered in real-world applications. Local search, by embracing heuristics and probabilistic choices, sacrifices completeness but gains immense scalability.

WalkSAT stands as a testament to the power of intelligent algorithmic design. By focusing on the efficiency of individual search steps and leveraging insights into the behavior of the search process near optima, it transformed the landscape of SAT solving. Its ability to tackle problems with hundreds of thousands of variables has made it an indispensable tool in various domains, from formal verification to planning and scheduling. This concludes our exploration of logic in AI, demonstrating how theoretical foundations, combined with clever algorithmic engineering, can lead to powerful practical solutions for some of the most challenging computational problems.